Coding Assignments

Six graded assignments with an automatic test suite

```
$ python grade.py
A1 Vectors & matrices   10.0 / 10
A2 Orthogonality & QR   15.0 / 15
A3 Eigenvalues          15.0 / 15
A4 The SVD              25.0 / 25
A5 Pseudo-inverse       20.0 / 20
A6 Applications         15.0 / 15
TOTAL                  100.0 / 100
```

Part 3 of a three-part course

1. **Textbook** — theory, worked hand calculations, exercises with full solutions.
2. **Python Lab Manual** — every algorithm implemented from scratch in Python, with real output.
3. **Coding Assignments** (this handout) — six graded assignments with an automatic `pytest` test suite.

Edition of October 2026

Contents

1 Getting Started **1**

1.1 What you will build . . . 1

1.2 Setting up . . . 1

1.3 The rules . . . 2

1.4 Running the tests . . . 2

1.5 Grading . . . 3

1.6 How to work . . . 3

2 A1 — Vectors and Matrices **4**

3 A2 — Orthogonality and QR **6**

4 A3 — Eigenvalues of Symmetric Matrices **8**

5 A4 — The Singular Value Decomposition **10**

6 A5 — The Pseudo-Inverse and Least Squares **12**

7 A6 — Applications **14**

A Troubleshooting **16**

B The Test Files **17**

CHAPTER 1

Getting Started

1.1 What you will build

Across six assignments you write a small numerical linear-algebra library: vector and matrix basics, Gaussian elimination, Gram–Schmidt and Householder QR, the power method and the Jacobi eigenvalue algorithm, two SVD algorithms, the pseudo-inverse, three least-squares solvers, regularisation, low-rank approximation and PCA. By the end, every function you call is one you wrote — and an automatic test suite has checked it against the textbook's hand calculations, against NumPy, and on the awkward cases (zero matrices, rank deficiency, wide and tall shapes, repeated singular values, ill-conditioning).

1.2 Setting up

You need Python 3.10 or newer. Unzip `svd_assignments_starter.zip`, then

```
cd svd-assignments
python -m pip install -r requirements.txt
python grade.py
```

A fresh copy prints a score of **0 / 100**, with every test listed as "not done". The folder contains:

```
svd_assignments/      YOUR CODE: six modules with functions marked TODO
    a1_vectors.py  a2_orthogonal.py  a3_eigen.py
    a4_svd.py      a5_pinv.py        a6_applications.py
tests/                the test suite (221 tests) -- do not edit
grade.py              runs the tests and prints your score per assignment
README.md, requirements.txt, pytest.ini, ASSIGNMENTS.pdf (this handout)
```

1.3 The rules

1. **No `numpy.linalg`, no `scipy`** (nor scikit-learn, PyTorch, JAX ...) inside `svd_assignments/`. This is enforced twice: a test reads your source code and rejects any such import or any use of `.linalg`; and while the tests run, every `numpy.linalg` function is replaced by one that raises `ForbiddenFunctionError`.
2. NumPy arrays, slicing, broadcasting and element-wise arithmetic (`+ - * /`, `np.sqrt`, `np.sum`, `np.abs`, `np.outer`, ...) are allowed everywhere.
3. In A1, `dot` and `matmul` must not use `@`, `np.dot`, `np.matmul`, `np.inner`, `np.einsum`, `np.tensordot` or `np.vdot`: those are what you are implementing. From A2 on, `@` is allowed.
4. Later assignments may import your earlier functions (`from .a3_eigen import jacobi_eigh`). Do the assignments in order: A5 and A6 build on your SVD from A4.
5. The **docstring of each function is its specification**: argument shapes, return conventions, error cases and tolerances. Read it before you start.
6. Do not edit the tests.

1.4 Running the tests

```
python grade.py                  # score report for all six assignments
python grade.py a3               # only Assignment 3
python grade.py a4 -v            # Assignment 4 with the failure messages
pytest tests/test_a2_orthogonal.py -x     # stop at the first failure
pytest -k "jacobi and not svd" -v         # tests selected by name
```

A score report looks like this (for a student who has finished A1 and A2):

```
======================== SVD assignments: score report ========================
Assignment                              passed  failed  not done        points
A1  Vectors & matrices                      35       0         0     10.0 / 10
A2  Orthogonality & QR                      31       0         0     15.0 / 15
A3  Eigenvalues                              0       0        24      0.0 / 15
A4  The SVD                                  0       0        76      0.0 / 25
A5  Pseudo-inverse & least squares           0       0        31      0.0 / 20
A6  Applications                             0       0        24      0.0 / 15
------------------------------------------------------------------------------
TOTAL                                                                25.0 / 100
```

"Not done" counts tests that hit `NotImplementedError`. Within an assignment every test carries equal weight. The "rules" test of each module (no forbidden imports) only scores once every function in that module is implemented.

1.5 Grading

	Assignment	Tests	Points	Textbook / Lab
A1	Vectors and matrices	35	10	Ch. 1–2 / Part 1
A2	Orthogonality and QR	31	15	Ch. 3 / Part 2
A3	Eigenvalues of symmetric matrices	24	15	Ch. 4–5 / Part 3
A4	The singular value decomposition	76	25	Ch. 6–7 / Part 4
A5	Pseudo-inverse and least squares	31	20	Ch. 8–9 / Parts 5–6
A6	Applications	24	15	Ch. 7, 10 / Parts 6–7
	Total	**221**	**100**	

1.6 How to work

- **Hand first, code second.** Before coding a function, do one small example by hand (the textbook's worked examples are ideal). Many tests use exactly those examples, so you will recognise the numbers.
- **One function at a time.** Run `pytest -k <function_name>` until it passes, then move on.
- **Try before you look.** The Lab Manual contains implementations of most of these algorithms. Attempt each function from the textbook and your own understanding first; consult the Lab Manual only when stuck, and then close it again and write your own version. Several functions here (`four_subspaces`, `classify_definiteness`, `eckart_young_error`, `complete_basis`, ...) are not in the Lab Manual at all.
- **Read failure messages.** The tests are written to say *what* is wrong ("R must be upper triangular", "columns of U are not orthonormal: $\max|Q^TQ - I| = 3.1\text{e-}04$").
- A separate archive, `svd_assignments_solutions.zip`, contains reference solutions. Keep it closed until you have a passing (or abandoned) attempt.

CHAPTER 2

A1 — Vectors and Matrices

File: `svd_assignments/a1_vectors.py` **Points:** 10 **Reading:** Textbook Ch. 1–2, Lab Part 1.

Goals

Implement the basic operations every later algorithm rests on, paying attention to two numerical details: overflow in norms and pivoting in elimination.

Function	What it must do
`dot(u, v)`	$\sum u_i v_i$ as a Python `float`; `ValueError` for different lengths. No `@`/`np.dot`.
`norm(x, p=2)`	$p \in \{1, 2, \infty\}$, `ValueError` otherwise. The 2-norm must not overflow for entries $\sim 10^{200}$ nor underflow for $\sim 10^{-200}$.
`normalize(x)`	$\boldsymbol{x} / \|\boldsymbol{x}\|$; `ValueError` for the zero vector.
`angle(u, v)`	Angle in radians in $[0, \pi]$; never `nan`; `ValueError` for a zero vector.
`project_onto_vector(b, a)`	$(\boldsymbol{a} \cdot \boldsymbol{b} / \boldsymbol{a} \cdot \boldsymbol{a})\, \boldsymbol{a}$; `ValueError` if $\boldsymbol{a} = \boldsymbol{0}$.
`outer(u, v)`	The $(\text{len}\, u \times \text{len}\, v)$ matrix $\boldsymbol{u}\boldsymbol{v}^{\mathsf{T}}$.
`matmul(A, B)`	AB with loops; `ValueError` on mismatched inner dimensions. No `@`/`np.dot`/`np.matmul`.
`rank_gaussian(A, tol=1e-10)`	Rank by Gaussian elimination *with partial pivoting*; entries $\le$ `tol`$\cdot \max(1, \max \|a_{ij}\|)$ count as zero.

Hints.

- For the overflow-safe 2-norm compute $s = \max_i |x_i|$ and return $s\sqrt{\sum (x_i/s)^2}$.
- `np.arccos` returns `nan` for arguments like 1.0000000000000002. Clip the cosine to $[-1, 1]$.
- For `rank_gaussian` you do not need the full reduced echelon form: count pivots during forward elimination. Partial pivoting means: in each column pick the remaining row with the largest |entry|.

What the tests check.

Textbook values (Worked Examples 1.1–1.4, 2.2, 2.3; Exercises 1.2, 2.3); random comparisons with NumPy; error cases; the $10^{\pm 200}$ overflow/underflow cases; that `angle` of par-

allel vectors is 0 and not `nan`; matrix products for shapes from 1×1 to 5×1 times 1×5; ranks of matrices that need a row swap, that need pivoting for accuracy (`[[1e-14, 1], [1, 1]]`), and random matrices of known rank; and that `dot` and `matmul` contain no built-in products.

Stretch goals (not graded).

Time your `matmul` against `@` for $n = 50, 100, 200$ and fit the exponent of n. Then write a version that loops only over k and adds outer products (view 4 of the textbook). How much faster is it, and why?

CHAPTER 3

A2 — Orthogonality and QR

File: `svd_assignments/a2_orthogonal.py` **Points:** 15 **Reading:** Textbook Ch. 3, Lab Part 2.

Goals

Orthonormalise vectors stably, factor $A = QR$ two ways, and build the tools the SVD needs later (back substitution, projections, completing a basis).

Function	What it must do
`gram_schmidt(A)`	Reduced QR by *modified* Gram–Schmidt: Q $(m \times n)$ orthonormal, R $(n \times n)$ upper triangular with *positive* diagonal. `ValueError` if a column's remaining part has norm $\leq 10^{-12}\times$ its original norm.
`qr_householder(A)`	Full QR of any $m \times n$ matrix by Householder reflections: Q $(m \times m)$ orthogonal, R $(m \times n)$ with exact zeros below the diagonal.
`back_substitution(U, b)`	Solve $Ux = b$, U upper triangular; `ValueError` on a zero diagonal entry.
`is_orthonormal(Q, tol=1e-10)`	`True` iff $\max \lvert Q^{\mathsf{T}}Q - I \rvert \leq$ `tol`.
`projection_matrix(A)`	P onto $\mathcal{C}(A)$ (independent columns): $P = P^{\mathsf{T}} = P^2$, $PA = A$.
`complete_basis(Q)`	Extend $m \times k$ orthonormal columns $(0 \leq k \leq m)$ to an $m \times m$ orthogonal matrix whose first k columns are exactly Q.

Hints.

- Classical and modified Gram–Schmidt differ by one variable: `R[i, j] = Q[:, i] @ A[:, j]` versus `R[i, j] = Q[:, i] @ w`. One of them fails the ill-conditioning test.
- Householder: for $x =$ `R[k:, k]`, use $v = x + \operatorname{sign}(x_1)\,\|x\|\,e_1$ and apply $H = I - 2vv^{\mathsf{T}}/v^{\mathsf{T}}v$ to `R[k:, :]` without forming H. Accumulate Q by applying the same reflection to the columns `Q[:, k:]`.
- `complete_basis`: try $e_1, e_2, \ldots$ in turn; subtract their components along the columns you have (do it twice); keep the result if its norm exceeds a small tolerance. Handle $k = 0$ (an input of shape `(m, 0)`).

What the tests check.

The QR of textbook Worked Example 3.3 entry by entry; orthonormality, triangularity, positivity and $QR = A$ for many shapes (including 1×1, $m \times 1$, wide matrices and rank-deficient or zero matrices for Householder); a matrix with $\kappa = 10^7$, where `gram_schmidt` must keep $\max |Q^\mathsf{T}Q - I| < 10^{-7}$ (classical Gram–Schmidt gives about 10^{-4}); a matrix with $\kappa = 10^{14}$, where Householder must stay orthogonal to 10^{-12}; the projection matrix of Worked Example 3.1; and basis completion for $k = 0$, $k = m$ and in between.

Stretch goals (not graded).

Add column pivoting to `qr_householder` (at step k, swap in the remaining column of largest norm). The diagonal of R then reveals the numerical rank. Compare it with the singular values on a rank-deficient matrix.

CHAPTER 4

A3 — Eigenvalues of Symmetric Matrices

File: `svd_assignments/a3_eigen.py` **Points:** 15 **Reading:** Textbook Ch. 4–5, Lab Part 3.

Goals

Compute eigenvalues the simple way (power iteration) and the complete, accurate way (Jacobi), and use them to classify quadratic forms.

Function	What it must do
`rayleigh_quotient(S, x)`	$\boldsymbol{x}^\mathsf{T} S\boldsymbol{x} / \boldsymbol{x}^\mathsf{T}\boldsymbol{x}$; `ValueError` for $\boldsymbol{x} = \mathbf{0}$.
`power_iteration(S, x0=None, tol=1e-12, max_iter=10000)`	Returns `(lam, v)`: dominant eigenvalue and a *unit* eigenvector.
`jacobi_eigh(S, tol=1e-14, max_sweeps=60)`	All eigenpairs of a symmetric matrix: eigenvalues in *decreasing* order, V orthogonal, $SV = V\Lambda$. `ValueError` if S is not square or not symmetric.
`spectral_reconstruct(eigvals, V)`	$V\,\mathrm{diag}(\lambda)\,V^\mathsf{T}$.
`classify_definiteness(S, tol=1e-10)`	One of `"positive definite"`, `"positive semidefinite"`, `"negative definite"`, `"negative semidefinite"`, `"indefinite"`. The zero matrix is positive semidefinite.

Hints.

- Jacobi rotation for the pair (p, q): $\vartheta = (s_{qq} - s_{pp})/(2s_{pq})$, $t = \mathrm{sign}(\vartheta)/(|\vartheta| + \sqrt{\vartheta^2 + 1})$, $c = 1/\sqrt{1+t^2}$, $s = ct$. Update columns p, q (`A[:, p] = c Ap - s Aq`, `A[:, q] = s Ap + c Aq`), then rows p, q the same way, then the columns of V.
- Stop when the off-diagonal norm is below `tol` times $\|S\|_F$. Compute that norm by summing the off-diagonal squares *directly* — `sum(S**2) - sum(diag**2)` cancels catastrophically (see the pitfall box in Lab Part 3), and a test with eigenvalues $1, 10^{-4}, 10^{-8}$ will catch it.
- `classify_definiteness` is three lines once you have `jacobi_eigh`: count eigenvalues above $+\tau$ and below $-\tau$.

What the tests check.

The Rayleigh quotients of Worked Example 4.4; power iteration on $\left[\begin{smallmatrix} 2 & 1 \\ 1 & 2 \end{smallmatrix}\right]$ and on a 6×6 matrix with known spectrum; Jacobi on the repeated-eigenvalue matrix of Worked Example 4.3, on random symmetric matrices of sizes $1, 2, 5, 12$ (compared with NumPy to 10^{-12}), on diagonal and indefinite matrices, and on a matrix with eigenvalues $1, 10^{-4}, 10^{-8}$ (absolute accuracy 10^{-14}); error cases; reconstruction; and seven definiteness classifications from textbook Chapter 5, plus $A^{\mathsf{T}}A$ versus AA^{T} for a wide A.

Stretch goals (not graded).

Implement the *threshold* Jacobi variant that skips rotations when $|s_{pq}|$ is below a sweep-dependent threshold, and compare rotation counts. Or implement inverse iteration with a shift to find the eigenvalue closest to a given number.

CHAPTER 5

A4 — The Singular Value Decomposition

File: `svd_assignments/a4_svd.py` **Points:** 25 **Reading:** Textbook Ch. 6–7, Lab Part 4.

Goals

Implement the SVD twice — the textbook recipe via $A^\mathsf{T}A$, and one-sided Jacobi, which never forms $A^\mathsf{T}A$ — and use it to answer questions about rank and the four fundamental subspaces.

Conventions (same as `numpy.linalg.svd`)

Return `(U, s, Vt)`: `s` is a 1-D array of the $k = \min(m, n)$ singular values in decreasing order, and `Vt` is V^T. With `full_matrices=False`, U is $m \times k$ and V^T is $k \times n$; with `True`, U is $m \times m$ and V^T is $n \times n$. Singular vectors are tested by their properties (orthonormality, reconstruction), never entry by entry, because of the sign freedom.

Function	What it must do
`svd_via_eig(A, full_matrices=False)`	Eigen-decompose $A^\mathsf{T}A$ (or AA^T for wide A) with your `jacobi_eigh`; $\sigma_i = \sqrt{\lambda_i}$; $\boldsymbol{u}_i = A\boldsymbol{v}_i/\sigma_i$; complete U. Eigenvalues $\le \max(m,n)\,\varepsilon\,\lambda_1$ count as zero and their σ_i must be exactly 0.
`svd_jacobi(A, full_matrices=False, tol=1e-15, max_sweeps=60)`	One-sided Jacobi on the columns of A (no $A^\mathsf{T}A$). Singular values $\le \max(m,n)\,\varepsilon\,\sigma_1$ must be exactly 0.
`singular_values(A)`	The singular values, decreasing.
`svd_reconstruct(U, s, Vt)`	$U\Sigma V^\mathsf{T}$ for reduced or full factors.
`matrix_rank(A, tol=None)`	Number of $\sigma_i >$ `tol`; default `tol` $= \max(m,n)\,\varepsilon\,\sigma_1$; the zero matrix has rank 0.
`four_subspaces(A, tol=None)`	Dict with keys `"row"` $(n \times r)$, `"null"` $(n \times (n-r))$, `"column"` $(m \times r)$, `"left_null"` $(m \times (m-r))$, each with orthonormal columns.

Hints.

- Handle a wide matrix by decomposing A^T and swapping: if $A^\mathsf{T} = U'\Sigma V'^\mathsf{T}$ then $A = V'\Sigma U'^\mathsf{T}$.

- One-sided Jacobi: for columns $\boldsymbol{w}_p, \boldsymbol{w}_q$ of the working matrix W (initially A, with $V = I$) let $\alpha = \|\boldsymbol{w}_p\|^2$, $\beta = \|\boldsymbol{w}_q\|^2$, $\gamma = \boldsymbol{w}_p \cdot \boldsymbol{w}_q$. Skip if $|\gamma| \leq \texttt{tol}\sqrt{\alpha\beta}$; otherwise rotate with $\zeta = (\beta - \alpha)/(2\gamma)$, $t = \operatorname{sign}(\zeta)/(|\zeta| + \sqrt{1+\zeta^2})$, applying the same rotation to W and V. Stop after a sweep with no rotation. Then $\sigma_j = \|\boldsymbol{w}_j\|$; sort; normalise.
- For rank-deficient A you cannot normalise the zero columns of W. Build U from the non-zero ones and fill the rest with your `complete_basis` from A2.
- `four_subspaces` is a few lines with the *full* SVD (textbook Section 6.6).

What the tests check.

The singular values of Worked Examples 6.1–6.4 and Exercise 6.1, for both methods; for both methods and both `full_matrices` settings, nine shapes including 1×1, tall, wide, square, rank-deficient and the zero matrix: shapes, non-negativity and ordering of s, orthonormality of U and V, reconstruction, and agreement with NumPy; exact zeros beyond the rank; the null-space vector of Worked Example 6.2; **accuracy**: one-sided Jacobi must find $\sigma_2 = 10^{-8}$ of the Läuchli matrix to 10^{-6} relative accuracy, and all singular values of a matrix with $\kappa = 10^{10}$; repeated singular values; reconstruction from reduced and full factors; ranks, including the `tol` argument; and all properties of the four subspaces (dimensions, orthonormality, $A \cdot \text{null} = 0$, $A^{\mathsf{T}} \cdot \text{left null} = 0$, spanning).

Stretch goals (not graded).

Implement Golub–Kahan bidiagonalisation (Householder from the left and the right) followed by one-sided Jacobi on the bidiagonal matrix, and compare speed with your plain one-sided Jacobi on 200×100 matrices. Or implement a randomised SVD for the top k singular triplets.

CHAPTER 6

A5 — The Pseudo-Inverse and Least Squares

File: `svd_assignments/a5_pinv.py` **Points:** 20 **Reading:** Textbook Ch. 8–9, Lab Parts 5–6.

Goals

Compute A^+ robustly and by the full-rank formulas, verify it with the Penrose conditions, and solve least-squares problems three ways.

Function	What it must do
`pinv(A, rcond=None)`	$V\Sigma^+U^\mathsf{T}$; singular values $\le$ `rcond`$\cdot\sigma_1$ are dropped (default `rcond` $= \max(m,n)\,\varepsilon$). Shape $n \times m$; zero matrix $\mapsto$ zero matrix.
`pinv_full_rank(A)`	$(A^\mathsf{T}A)^{-1}A^\mathsf{T}$ or $A^\mathsf{T}(AA^\mathsf{T})^{-1}$ (solve with your QR); `ValueError` if rank-deficient.
`penrose_residuals(A, X)`	Tuple of four floats: $\|AXA - A\|_F$, $\|XAX - X\|_F$, $\|(AX)^\mathsf{T} - AX\|_F$, $\|(XA)^\mathsf{T} - XA\|_F$.
`cholesky(S)`	Lower-triangular L with $LL^\mathsf{T} = S$; `ValueError` if S is not positive definite.
`lstsq_normal(A, b)`	Normal equations solved with your Cholesky; `ValueError` for dependent columns.
`lstsq_qr(A, b)`	$R_1\boldsymbol{x} = (Q^\mathsf{T}\boldsymbol{b})_1$ with Householder QR (independent columns).
`lstsq_min_norm(A, b, rcond=None)`	$A^+\boldsymbol{b}$ — works for every A.

Hints.

- `(Vt.T * s_inv) @ U.T` computes $V\Sigma^+U^\mathsf{T}$ without building Σ^+: broadcasting scales column i of V by $1/\sigma_i$.
- Cholesky, column by column: $l_{jj} = \sqrt{s_{jj} - \sum_{k<j} l_{jk}^2}$ and $l_{ij} = (s_{ij} - \sum_{k<j} l_{ik}l_{jk})/l_{jj}$ for $i > j$. If the quantity under the square root is ≤ 0, the matrix is not positive definite.
- A `_solve(M, B)` helper using your QR is provided in the starter file.

What the tests check.

The pseudo-inverses of Worked Examples 9.1–9.5 and Exercises 9.1–9.2; zero and 1×1 matrices; random tall, wide, square and rank-deficient matrices against NumPy, with the Penrose residuals below 10^{-10}; the effect of `rcond`; $(A^+)^+ = A$ and $(A^\mathsf{T})^+ = (A^+)^\mathsf{T}$; that the Penrose residuals detect a wrong answer; the full-rank formulas and their error case; Cholesky; the line fit of Worked Example 8.1 with all three solvers; random least-squares problems (residual orthogonal to the columns); a degree-10 polynomial fit that QR must solve to 10^{-6}; and minimum-norm solutions for underdetermined and rank-deficient systems (Worked Examples 8.4, 9.4; Exercise 8.2).

Stretch goals (not graded).

Write `lstsq_general_solution(A, b)` returning $\boldsymbol{x}^+$ and an orthonormal basis N of $\mathcal{N}(A)$, so that every least-squares solution is $\boldsymbol{x}^+ + N\mathbf{0}$. Check that all of them have the same residual and that $\boldsymbol{x}^+$ is the shortest.

CHAPTER 7

A6 — Applications

File: `svd_assignments/a6_applications.py` **Points:** 15 **Reading:** Textbook Ch. 7 and 10, Lab Parts 6–7.

Goals

Put the SVD to work: measure matrices, compress them optimally, regularise ill-posed problems and analyse data.

Function	What it must do
`spectral_norm(A)`, `frobenius_norm(A)`, `nuclear_norm(A)`	σ_1, $\sqrt{\sum a_{ij}^2}$, $\sum \sigma_i$.
`condition_number(A)`	σ_1/σ_r with σ_r the smallest singular value above $\max(m,n)\,\varepsilon\,\sigma_1$; `np.inf` for the zero matrix.
`low_rank_approx(A, k)`	$\sum_{i\le k} \sigma_i \boldsymbol{u}_i \boldsymbol{v}_i^{\mathsf{T}}$; $k=0$ gives zeros, $k \ge \min(m,n)$ gives A; `ValueError` for $k<0$.
`eckart_young_error(s, k, norm="fro")`	From the singular values alone: σ_{k+1} (`"2"`) or $\sqrt{\sum_{i>k}\sigma_i^2}$ (`"fro"`); 0 when $k \ge$ `len(s)`; `ValueError` for another norm.
`compression_ratio(m, n, k)`	$k(m+n+1)/(mn)$.
`truncated_svd_solve(A, b, k)`	$\sum_{i\le k}(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{b}/\sigma_i)\boldsymbol{v}_i$; `ValueError` unless $1 \le k \le r$.
`ridge_solve(A, b, lam)`	$\sum \frac{\sigma_i}{\sigma_i^2+\lambda}(\boldsymbol{u}_i^{\mathsf{T}}\boldsymbol{b})\boldsymbol{v}_i$; $\lambda = 0$ must give $A^{+}\boldsymbol{b}$ even for rank-deficient A; `ValueError` for $\lambda < 0$.
`pca(X, k)`	`(components, explained_variance, scores, mean)` for the rows of X: components $k \times d$, variances $\sigma_i^2/(N-1)$, scores $(X-\bar{x})\,\text{components}^{\mathsf{T}}$, mean of length d.

Hints.

- Most of these are two or three lines on top of your `svd_jacobi`.
- In `ridge_solve`, compute the filter $\sigma_i/(\sigma_i^2+\lambda)$ only where $\sigma_i > 0$, so that $\lambda = 0$ does not divide by zero.
- Do not forget to centre the data in `pca`.

What the tests check.

The norms of Worked Example 7.2 and random comparisons with NumPy; condition numbers (Worked Example 7.4, rank-deficient and zero matrices); the best rank-one approximations of Worked Example 7.3 and Exercise 7.2; the Eckart–Young errors in both norms for $k = 0, 1, 3, 5, 9$; Exercise 7.4 for `eckart_young_error`; Exercise 7.3 for `compression_ratio`; Worked Example 10.2 for TSVD; Worked Example 10.3 and Exercise 10.2 for ridge, agreement with $(A^{\mathsf{T}}A + \lambda I)^{-1}A^{\mathsf{T}}\boldsymbol{b}$, the $\lambda \to 0$ limit and the monotone decrease of $\|\boldsymbol{x}_\lambda\|$; and PCA on Worked Example 10.4 and on a random 4-dimensional cloud.

Stretch goals (not graded).

Apply `low_rank_approx` to a real photograph (converted to greyscale) and plot the relative error against the compression ratio. Then implement generalised cross-validation (GCV) to choose λ in `ridge_solve` automatically, and test it on the deblurring problem of Lab Part 6.

APPENDIX A

Troubleshooting

`ModuleNotFoundError: No module named 'svd_assignments'` Run `pytest` or `python grade.py` from the top-level folder (the one containing `pytest.ini`).

`ForbiddenFunctionError: numpy.linalg.svd was called` Some code path calls `numpy.linalg`. Replace it with your own function.

`test_no_forbidden_imports` fails with a line number Your source imports `scipy`/`numpy.linalg` or uses `.linalg` somewhere, even in unused code. Remove it.

`TimeoutError: test took longer than 60 s` Usually an infinite loop: a `while` whose stopping test can never be met in floating point. Always add a maximum iteration count.

A test fails by a tiny amount, e.g. `3.1e-11 > 1e-12` Accuracy matters here. Typical causes: classical instead of modified Gram–Schmidt; computing a small quantity as a difference of large ones; stopping an iteration too early; forming $A^{\mathsf{T}}A$ where it is not needed.

The tests pass one by one but not all together You are probably modifying an input array in place. Copy inputs first (`_as_float_matrix` returns a copy).

APPENDIX B

The Test Files

File	Tests	Covers
`tests/test_a1_vectors.py`	35	A1
`tests/test_a2_orthogonal.py`	31	A2
`tests/test_a3_eigen.py`	24	A3
`tests/test_a4_svd.py`	76	A4
`tests/test_a5_pinv.py`	31	A5
`tests/test_a6_applications.py`	24	A6
`tests/conftest.py`	—	blocks `numpy.linalg`, time limit, score report
`tests/_helpers.py`	—	reference functions and shared checks

Reading the tests is allowed and encouraged: each test is a precise statement of a property your function must have, and many use the textbook's worked examples as data.